Review plan · for the teacher

Solving school problems in Python: unit check

Optimization · Level 3 · Unit check

Time	Materials
35 min	Board, chalk, paper. No devices needed.

Learning goals

- A timetable that works
- Exams without clashes
- The school bus route
- Matching teachers and classes

Activity: Model it in teams

1. Warm-up (5 min): For each of the four lessons, the class names one real decision at their school: a timetable, exam week, a route, sharing out jobs.
2. Model (15 min): Each team takes one decision and writes, on paper, its options (variables), its rules (constraints) and its goal, as plainly as they can, with a line of Python where they can.
3. Swap (10 min): Teams swap papers. Each team looks for one missing rule, the kind a solver would never guess, and writes it down.
4. Wrap-up (5 min): For each model, ask who the best plan might be hardest on, and what rule or second goal would protect them.

No devices needed. If you have a phone, run one team's model in the "Model it with CP-SAT" lab at the end.

Discussion

- Which school decision would you model first, and why?
- When is a quick rule good enough?
- Who should see a model's rules before it decides?

Exit ticket

- One school decision I will model, its goal, and one fairness rule.

Try it at school tomorrow
Pick one decision at your school this term (duties, exam slots, the bus, who teaches what). With two colleagues, write its options, rules and goal, model the smallest version in the "Model it with CP-SAT" lab, and show the result to the people it affects.

Quiz sheet · for each team

Solving school problems in Python: unit check

Optimization · Level 3 · Unit check

Team _____ Date _____

1 **In a CP-SAT timetable model, what is each lesson's variable?**

◯ The period it goes in ◯ The number of pupils in the class ◯ Its teacher's name

2 **INFEASIBLE means the solver ran out of time before finding a timetable.**

◯ True ◯ False

3 **The second rule should stop a teacher (one per subject) being in two classes at once, but timetables still clash. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  for c in classes:
2      m.add_all_different([p[c, s] for s in subjects])
3  for s in subjects:
4      m.add_all_different([p[c, s] for s in subjects])
```

4 **Match each school problem to the kind of problem it is.**

Exam slots without clashes	• • • •	Travelling salesperson
The bus visiting every village	• • • •	An all-different rule
One teacher for each class	• • • •	Graph colouring
No class with two lessons at once	• • • •	Assignment

5 **Which of these prove that 3 exam slots is the fewest possible? Choose all that apply.**

☐ The solver says OPTIMAL when minimising the slots ☐ DSatur found a plan with 3 slots

☐ Three exams that all clash with each other ☐ Greedy colouring used 4 slots

6 **Put in order: building and improving a bus route.**

☐ Keep the change if the route is shorter

☐ Repeat until no swap helps

☐ Build a route with nearest neighbour

☐ Reverse the stretch between them

☐ Take out two roads

7 **What does this program print?**

```
d = [[0, 2, 4], [2, 0, 3], [4, 3, 0]]
route = [0, 1, 2]
back = route[1:] + [0]
print(sum(d[a][b] for a, b in zip(route, back)))
```

◯ 6 ◯ 5 ◯ 9

8 **Complete the habit.**

Judge a plan twice: by its _____ and by its _____ case, and agree the _____ with the people it affects.

Word bank: rule · period · worst · total · colour

9 **Complete the assignment model: exactly one class per teacher, and the best total score.**

```
for t in range(n):
    m._____(x[t])
for c in range(n):
    m.add_exactly_one([x[t][c] for t in range(n)])
m._____(sum(score[t][c] * x[t][c]
    for t in range(n) for c in range(n)))
```

Word bank: `add_all_different` `add_exactly_one` `minimize` `maximize`

10 **Put the lines of nearest neighbour in order.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

☐ `while len(route) < n:`

☐ `route = [0]`

☐ `route.append(max(left, key=lambda v: last[v]))`

☐ `route.append(min(left, key=lambda v: last[v]))`

☐ `left = [v for v in range(n) if v not in route]`

☐ `last = d[route[-1]]`

Answers · for the teacher

Solving school problems in Python: unit check

Optimization · Level 3 · Unit check

1 **The period it goes in**

The solver decides when each lesson happens, so the period is the variable.

2 **False**

INFEASIBLE is a proof that no timetable keeps all the rules.

3 **Line 4: `m.add_all_different([p[c, s] for c in classes])`**

For each subject, the rule must cover that subject's lessons in every class: loop over the classes.

4 **Exam slots without clashes — Graph colouring**
The bus visiting every village — Travelling salesperson
One teacher for each class — Assignment
No class with two lessons at once — An all-different rule

Naming the kind of problem tells you which methods and labs help: colouring for clashes, routes for the bus, assignment for pairing, and all-different rules for timetables.

5 **Three exams that all clash with each other; The solver says OPTIMAL when minimising the slots**

A clique of three needs three slots in any plan, and OPTIMAL means the solver proved nothing smaller works.

6 **Top to bottom: 4, 5, 1, 3, 2**

A quick rule builds a whole route first; 2-opt then improves it one swap at a time and stops when no single swap helps, which may still be short of the best.

7 **9**

zip pairs each stop with the next, and back ends with the school: 0→1 (2) + 1→2 (3) + 2→0 (4) = 9.

8 **Judge a plan twice: by its <u>total</u> and by its <u>worst</u> case, and agree the <u>rule</u> with the people it affects.**

A plan with the best total can still be hard on one person: the longest bus ride, three exams in a row, the worst match.

9

```
for t in range(n):
    m.add_exactly_one(x[t])
for c in range(n):
    m.add_exactly_one([x[t][c] for t in range(n)])
m.maximize(sum(score[t][c] * x[t][c]
    for t in range(n) for c in range(n)))
```

Each row of yes-or-no variables needs exactly one yes, like each column, and the goal is the largest total score, so maximise.

10 **Top to bottom: 2, 1, ✗, 5, 4, 3**

```
route = [0]
while len(route) < n:
    last = d[route[-1]]
    left = [v for v in range(n) if v not in route]
    route.append(min(left, key=lambda v: last[v]))
```

Start at the school, then, while places are left, look at the distances from the last stop and add the closest place not yet visited: min, not max.